

133. Ralentir sans compter les images à la main

`retard()` dit quand le tour de boucle a débordé.

1. Ce qu'il faut comprendre

`retard()` rend 1 si le tour précédent a duré **plus d'une image**. C'est la seule façon honnête de savoir qu'un jeu ne tient plus les soixante images par seconde : le compteur d'images de la page mesure le navigateur, pas la console.

Redessiner trois cent soixante tuiles à chaque tour est un bon moyen de faire monter ce compteur. **Un vrai jeu ne redessine que ce qui a changé** — et c'est exactement le débordement qui fait rater un `images() % 20 == 0`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1011 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         102 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles - la police entière
"RETARDS"      7 o ← texte() - un texte gravé
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EffacerCases`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 1    23 o sur place, dans main() + AdresseCase (10 o)
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 1   10 o sur place, dans main() + EcrireNombre (33 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
retard() × 1     2 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : redessine tout l'écran à chaque tour (trop de travail !),
// et compte les tours qui ont duré plus d'une image.
//
// retard() rend 1 si le tour d'avant a pris PLUS d'une image (1/60 de seconde).
// Un jeu qui prend du retard ralentit : un vrai jeu ne redessine
// que ce qui a changé.

#include <poser>    // pose une tuile sur une case du fond
#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    uint8_t sautees = 0;    // Le nombre de tours en retard.

    while (true) {          // Répète pour toujours :
        // Un travail volontairement lourd, pour voir la mesure bouger. La
        // première ligne est laissée au compte rendu : la redessiner l'effacerait
        // aussitôt, et l'on croirait que texte() ne marche pas.
        for (uint8_t y = 2; y < 18; y++)    // lignes 2 à 17...
            for (uint8_t x = 0; x < 20; x++) // ... toutes les colonnes :
                poser(x, y, (x + y) % 4);    // les tuiles 0, 1, 2, 3 à tour de rôle.
                                                // 16 x 20 = 320 tuiles à chaque tour.

        if (retard()) sautees++;    // Le tour d'avant a débordé ? On le compte.

        texte(1, 0, "RETARDS");
        nombre(10, 0, sautees);    // Le compteur monte : le travail est trop lourd.
        image();                    // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le compteur de retards monte : la boucle ne tient pas l'image.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'étiquette est écrite
- ✓ la boucle déborde vraiment (8 → 68)
- ✓ et le compteur est affiché

Ensuite : **134. Passer à la salle suivante, sans bouton** — chapitre 23 : Un vrai jeu. · gameboy3 — écrire une cartouche Game Boy en C++